Environment

- Install python form miniconda - https://docs.conda.io/projects/miniconda/en/latest/

- Create a project specific environment – conda create –name envname python=3.xx

- Create project directory – whatever

- Install the relevant base packages and software outside of python development
  - git – specific cases or latest or custom releases, cutimg edge versions
  - brew – packages outside python development e.g. OpenSSL, SQLite, GCC, PostgreSQL, Redis, Node.js wget, ffmpeg, sqlite3
  - pip – python only packages, does not manage system dependencies
  - conda – package and environment manager (bundles python, system and packages)

- **IDE's**
  - Jupyter Notebook. – pip install notebook
  - VSCode- https://code.visualstudio.com/
  - PyCharm - https://www.jetbrains.com/pycharm/
  - Cursor - https://www.cursor.com/
  - Colab - https://drive.google.com/drive/home
  - VSCode + Colab - Colab extensions (pay as you go)
  - ClaudeCode - https://claude.com/product/claude-code
  - Other….

- Thesis writing platform (LaTeX) - https://www.overleaf.com/

Environment - Python Package Installation & Version Compatibility

Multiple ways to install Python packages

- pip - pip alone does not track compatibility
- Conda - actually *solves* the dependency graph, base env
- Poetry - Enforces compatibility, Creates virtual environments

PIP

- **pip install pkg** - install something
- **pip install --upgrade pkg** - update something
- **pip show pkg** - see info about something
- **pip list** - show all installed packages
- **pip freeze** - output exact versions (for saving)
- **pip-compile** - solve + lock dependencies
- **pip-sync** - install only what's in the lock file

Check Environment

- **Pip freeze**
- **Pip list**

```
import pkgutil
common_ds = [
"numpy", "pandas", "matplotlib", "seaborn", "scipy", "sklearn",
"statsmodels", "plotly", "tensorflow", "torch", "xgboost",
"lightgbm", "pyarrow", "pydantic", "networkx", "nltk",
"spacy", "pillow", "joblib", "sympy"
]

installed = {p.name for p in pkgutil.iter_modules()}

[p for p in common_ds if p in installed]

  return go(f, seed, [])
}
```